

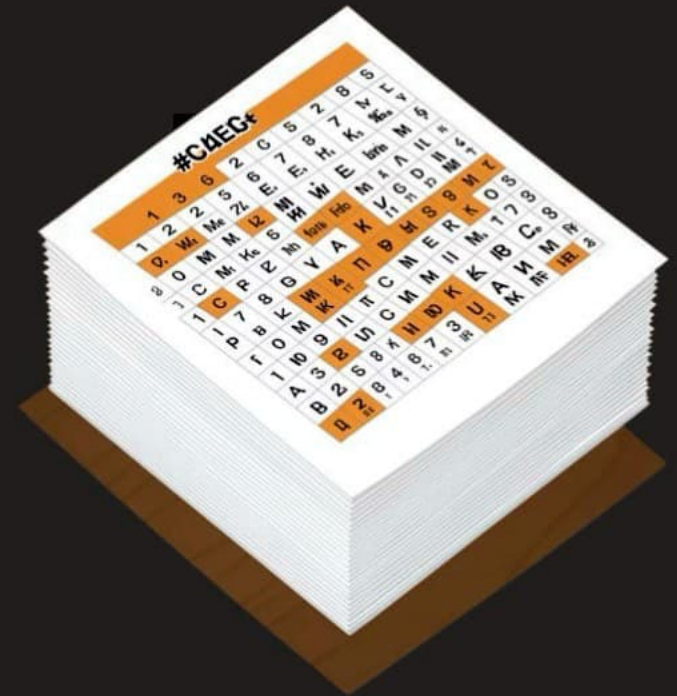
Word Search Generator

Project by: Saraf Salsabil & Mahbin Chowdhury

Group: 10

Course: Introduction to C Programming

Instructor: Prof. Mohammad Shifat-E-Rabbi



Introduction

What is a word search?

A square grid of letters where hidden words appear horizontally, vertically, or diagonally.

What this program does

Generates a letter grid, places words randomly in 8 directions, fills remaining cells, and lets the user search for words interactively.





Learning Objectives

Arrays

Work with 2D arrays to model the letter grid and tracking highlights.

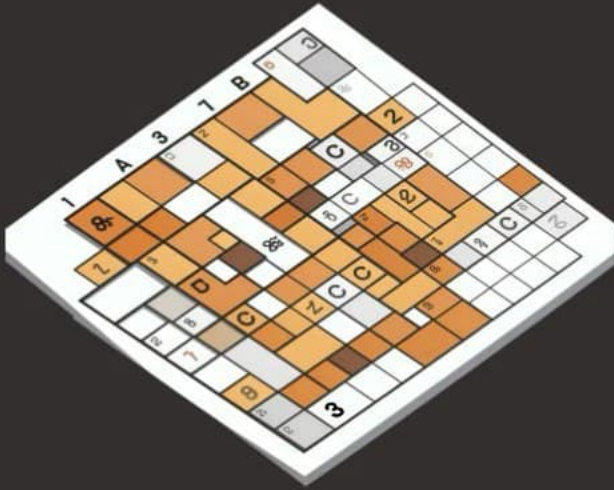
C Programming Practice

Use pointers, strings, standard libraries, and safe input handling.

Build a Small Game

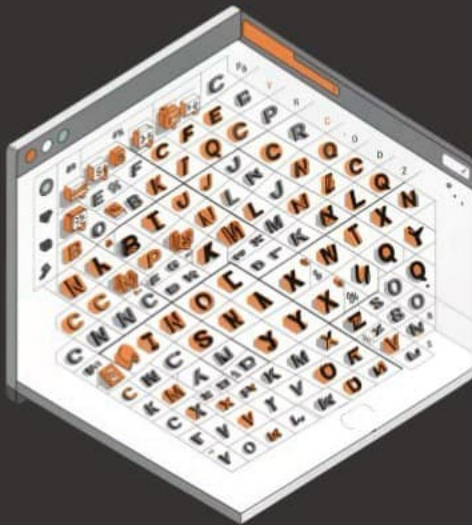
Create a simple interactive program that feels like a game and demonstrates program flow.

Key Features



- Generates an $N \times N$ grid (5–20) with placeholder cells.
- Places words in random positions and any of 8 directions.
- Fills empty cells with random uppercase letters A–Z.
- Searches input words and highlights the first match.

Concepts & Library Usage



- 2D arrays to store the grid: `char grid[MAX_SIZE][MAX_SIZE]`
- Loops (for/while) for placement, filling, and searching
- Functions for modularity: `init_grid`, `place_word`, `fill_grid`, `find_word`
- `rand()`, `srand()`, `time()` to seed and generate randomness

Program Flow (Most Important)

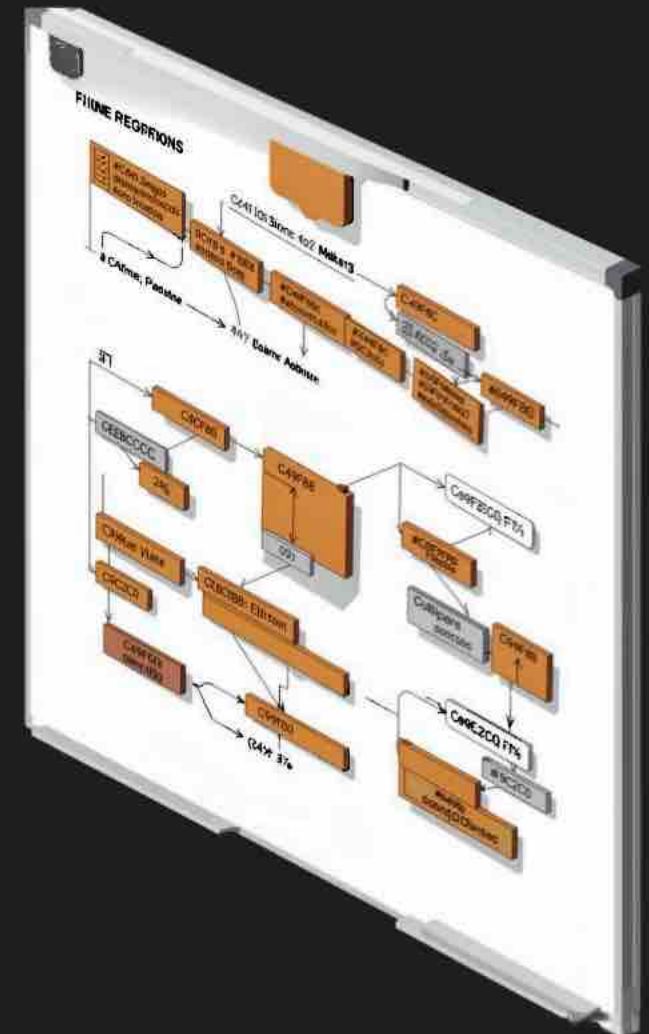
Init Grid

Place Words

Fill Blanks

Show Grid

Each step maps to a dedicated function or block in main(). Reliable placement and boundary checks are critical to avoid overlaps and out-of-bounds errors.



Placement & Search Details



01

Placement attempts

Try up to 1000 random start + direction combinations per word;
check boundary and overlap constraints before committing.

02

Overlap rule

Allow overlap only if existing letter matches the required letter for that position.

03

Search algorithm

From every cell, probe all 8 directions; if all letters match sequentially, mark cells as highlighted and return the first found occurrence.

Code Snapshot & Key Functions



init_grid()

Sets every cell to '.' to represent empty slots.



place_word()

Attempts random placements with boundary and overlap checks; returns success/failure.



fill_grid()

Replaces '.' with random letters 'A'-'Z' after placement.



find_word()

Searches all start positions and directions; sets highlight array for display.



Output & Conclusion



Example output: program prints the generated grid, then prompts for a word. If found, it shows the grid again with the matched letters highlighted (ANSI green in terminal).

What I learned: Practical use of 2D arrays, defensive placement logic, modular C functions, and designing simple interactive tools. Keep UI minimal and rely on clear terminal output.